POSTGRESQL

From Database Fundamentals to Advanced PostgreSQL Concepts

ABSTRACT

This course introduces PostgreSQL, covering database fundamentals, SQL, CRUD operations, data types, constraints, joins, functions, views, triggers, and other essential concepts with practical examples for beginners.

Sonu Suman

PostgreSQL Database Management System (DBMS)

Table of Contents

DATA :

Data is a collection of raw, unprocessed facts, figures, or observations that have not yet been organized or interpreted. By themselves, data may not have meaningful context, but they become useful when processed into information.

Data (Raw Facts)	Information (Processed Data)
Sonu, 20, CSE	Sonu is a 20-year-old Computer Science Engineering student.
95, 88, 91	Average marks = 91.3
1000, 1200, 1500	Sales are increasing over time.

INFORMATION:

Information is data that has been processed, organized, and interpreted to provide meaning and usefulness.

Student	Marks
Sonu	85
Rahul	90
Priya	95

DATABASE:

A database is an organized collection of data that is stored electronically and managed so it can be easily accessed, updated, and deleted.

Student ID	Name	Email	Phone
101	Sonu	sonu@email.com	9876543210
102	Rahul	rahul@email.com	9876501234

RDBMS:

RDBMS (Relational Database Management System) is a type of **Database Management System (DBMS)** that stores data in the form of **tables (relations)** consisting of **rows and columns**. It allows multiple tables to be related using **keys**, ensuring efficient storage, retrieval, and management of data.

Student_ID (PK)	Name	Department
101	Sonu	CSE

Student_ID (PK)	Name	Department
102	Rahul	ECE

Example of Database :

S.No.	Database Name	Type
1	MySQL	Relational (SQL)
2	PostgreSQL	Relational (SQL)
3	Oracle Database	Relational (SQL)
4	Microsoft SQL Server	Relational (SQL)
5	SQLite	Relational (SQL)
6	MongoDB	NoSQL (Document)
7	Redis	NoSQL (Key-Value)
8	Apache Cassandra	NoSQL (Wide-Column)
9	Neo4j	NoSQL (Graph)
10	Apache CouchDB	NoSQL (Document)

Installation of PostgreSql

1. Download the **PostgreSQL** installer from its official website.
2. Run the installer and follow the setup wizard to install PostgreSQL and its tools.
3. Set a password for the **postgres** user and choose the default port (**5432**).
4. Complete the installation and launch **pgAdmin** or the PostgreSQL command-line tool.
5. Verify the installation by connecting to the PostgreSQL server and creating a test database.

Command	Description
\l	List all databases (like SHOW DATABASES in MySQL).

Command	Description
\c database_name	Connect to a database.
\dt	Show all tables in the current database.
\d table_name	Describe the structure of a table.
\dn	List all schemas.
\du	List all users (roles).
\conninfo	Show current database connection information.
SELECT version();	Display the PostgreSQL version.
SELECT current_database();	Show the current database name.
SELECT current_user;	Show the current logged-in user.
CREATE DATABASE mydb;	Create a new database.
DROP DATABASE mydb;	Delete a database.
CREATE TABLE student(id SERIAL PRIMARY KEY, name VARCHAR(50));	Create a table.
INSERT INTO student(name) VALUES ('Sonu');	Insert a record.
SELECT * FROM student;	Display all records from the table.
UPDATE student SET name='Rahul' WHERE id=1;	Update a record.
DELETE FROM student WHERE id=1;	Delete a record.
DROP TABLE student;	Delete a table.
\q	Exit PostgreSQL (psql).
\?	Show all psql meta-commands.
\h	Show SQL command help.

Steps to Connect PostgreSQL in Command Prompt (CMD)

1. Add the PostgreSQL **bin** folder to the **Windows Environment Variables (PATH)**. Example:
2. C:\Program Files\PostgreSQL\17\bin

(Replace 17 with your installed PostgreSQL version if different.)

3. Open **Command Prompt (CMD)**.
4. Type the following command and press **Enter**:
5. psql -U postgres
6. When prompted, enter the **postgres** user password. *(The password will not be visible while typing—this is normal.)*
7. If the password is correct, you will see the PostgreSQL prompt:
8. postgres=#

You are now successfully connected to PostgreSQL.

What is CRUD?

CRUD stands for **Create, Read, Update, and Delete**. These are the four basic operations performed on data in a database.

Operation	Meaning	SQL Example
Create (C)	Add new data to the database.	INSERT
Read (R)	Retrieve or view data from the database.	SELECT
Update (U)	Modify existing data in the database.	UPDATE
Delete (D)	Remove data from the database.	DELETE

Here is the basic PostgreSQL SQL sequence to **create a database, create a table, insert data, view the schema, and show tables**.

1. Create a Database

CREATE DATABASE student_db;

2. Connect to the Database

\c student_db

3. Create a Table

```
CREATE TABLE students (
id SERIAL PRIMARY KEY,
name VARCHAR(50),
age INT,
course VARCHAR(50),
email VARCHAR(100)
);
```

4. Insert Information

```
INSERT INTO students (name, age, course, email)
VALUES
('Sonu', 20, 'CSE', 'sonu@gmail.com'),
('Rahul', 21, 'IT', 'rahul@gmail.com'),
('Priya', 19, 'ECE', 'priya@gmail.com');
```

5. Show All Tables

\dt

6. Show the Table Schema (Structure)

\d students

7. View the Data

SELECT * FROM students;

Expected Output

\dt

```
List of relations
Schema |   Name   | Type  |  Owner
--------+----------+-------+----------
public | students | table | postgres
```

\d students

Column | Type | Modifiers

--------+-----------------------+-------------------------

id | integer | PRIMARY KEY

name | character varying(50) |

age | integer |

course | character varying(50) |

email | character varying(100) |

These are the basic PostgreSQL commands most beginners use when learning CRUD operations.

What is a Table?

A **table** is a collection of related data organized into **rows and columns** in a database. Each **row** represents a record, and each **column** represents a specific attribute or field.

Example:

ID	Name	Age
1	Sonu	20
2	Rahul	21

Here are the **most commonly used UPDATE queries in PostgreSQL**, covering the main possibilities.

Syntax

UPDATE table_name

SET column_name = value

WHERE condition;

---

1. Update One Column

UPDATE students

```
SET age = 21
WHERE id = 1;
```

---

2. Update Multiple Columns

```
UPDATE students
SET name = 'Rahul',
age = 22
WHERE id = 2;
```

---

3. Update All Rows

```
UPDATE students
SET course = 'CSE';
```

Note: This updates every row in the table.

---

4. Update Using a Condition

```
UPDATE students
SET course = 'IT'
WHERE name = 'Sonu';
```

---

5. Update Using Multiple Conditions

```
UPDATE students
SET age = 23
WHERE course = 'CSE' AND name = 'Rahul';
```

---

6. Increase a Numeric Value

```
UPDATE students
SET age = age + 1
```

```
WHERE id = 1;
```

---

7. Update All Records Matching a Condition

```
UPDATE students
SET course = 'Computer Science'
WHERE course = 'CSE';
```

---

8. Verify the Update

```
SELECT * FROM students;
```

General Syntax

```
UPDATE table_name
SET column1 = value1,
column2 = value2
WHERE condition;
```

Important Notes

- **Always use a WHERE clause** when you want to update specific rows.
- If you **omit the WHERE clause**, **all rows** in the table will be updated.

Example

```
UPDATE students
SET age = 25;
```

This changes the age of **every student** to **25**.

Exam Definition

The UPDATE statement is used to modify existing records in a table. The WHERE clause is used to specify which record(s) should be updated.

DELETE Queries in PostgreSQL

The **DELETE** statement is used to remove records from a table.

Syntax

DELETE FROM table_name

WHERE condition;

---

1. Delete One Record

DELETE FROM students

WHERE id = 1;

Deletes the student whose id is 1.

---

2. Delete Using Name

DELETE FROM students

WHERE name = 'Sonu';

Deletes the record where the name is **Sonu**.

---

3. Delete Using Multiple Conditions

DELETE FROM students

WHERE age = 20 AND course = 'CSE';

Deletes records that match both conditions.

---

4. Delete Multiple Records

DELETE FROM students

WHERE age < 18;

Deletes all students whose age is less than 18.

---

5. Delete All Records (Keep the Table)

DELETE FROM students;

Deletes **all records** from the table, but the table structure remains.

---

6. Delete Using IN

DELETE FROM students

WHERE id IN (2, 4, 6);

Deletes students with IDs **2, 4, and 6**.

---

7. Delete Using LIKE

DELETE FROM students

WHERE name LIKE 'S%';

Deletes all students whose names start with **S**.

---

8. Verify the Deletion

SELECT * FROM students;

Displays the remaining records.

Delete Table (Structure + Data)

DROP TABLE students;

Deletes the **entire table**, including its structure and all data.

Delete Database

DROP DATABASE student_db;

Deletes the **entire database**.

Important Notes

- Use WHERE to delete specific records.
- If you omit the WHERE clause, **all records** in the table will be deleted.

- DROP TABLE removes the table itself.
- DROP DATABASE removes the entire database.

Exam Definition

The DELETE statement is used to remove one or more records from a table. The WHERE clause specifies which records should be deleted. If WHERE is omitted, all records in the table are deleted.

Drawbacks of a Poor Table Structure

1. **Duplicate IDs**
    - The same ID appears more than once.
    - It becomes difficult to uniquely identify each record.
    - Data integrity is affected.
2. **NULL Values**
    - Missing values (such as a NULL city) indicate incomplete data.
    - This can lead to inaccurate reports and query results.
3. **Data Redundancy**
    - Duplicate records or repeated information waste storage space.
    - Increases the chance of inconsistent data.
4. **Difficulty in Data Management**
    - Updating or deleting records becomes more complicated.
    - Duplicate records may cause accidental updates or deletions.
5. **Reduced Data Accuracy**
    - Duplicate and missing values reduce the reliability of the database.
    - Decision-making based on such data may be incorrect.

How to Avoid These Problems

- Use a **Primary Key** to ensure each record has a unique ID.
- Apply **NOT NULL** constraints for mandatory fields.
- Use **UNIQUE** constraints where duplicate values are not allowed.

- Normalize the database to reduce redundancy.

These are the common drawbacks of an improperly designed database table.

PostgreSQL Data Types – Complete Notes

What are Data Types?

Data types define the **type of data** that can be stored in a table column. They ensure that only valid and appropriate data is stored in the database.

Exam Definition

A data type specifies the kind of value that can be stored in a database column, such as numbers, text, dates, or Boolean values.

Why are Data Types Important?

- Ensure data is stored correctly.
- Prevent invalid data from being entered.
- Improve database performance.
- Reduce storage space.
- Maintain data consistency and integrity.

Common PostgreSQL Data Types

Data Type	Description	Example
INTEGER (INT)	Stores whole numbers	10, 500
SMALLINT	Stores small whole numbers	250
BIGINT	Stores very large whole numbers	9876543210
SERIAL	Auto-incrementing integer	1, 2, 3...
VARCHAR(n)	Stores variable-length text	'Sonu'
CHAR(n)	Stores fixed-length text	'A'
TEXT	Stores long text	'This is a paragraph.'

Data Type	Description	Example
BOOLEAN	Stores True or False values	TRUE, FALSE
DATE	Stores dates	2026-07-05
TIME	Stores time	10:30:00
TIMESTAMP	Stores both date and time	2026-07-05 10:30:00
DECIMAL(p,s) / NUMERIC(p,s)	Stores exact decimal numbers	99.95
REAL	Stores single-precision decimal values	3.14
DOUBLE PRECISION	Stores double-precision decimal values	12345.6789

Conditions for Common Data Types

1. INTEGER (INT)

Stores **whole numbers only**.

age INT

✓ 25

✓ 100

✗ 25.35

---

2. SERIAL

Automatically generates numbers.

id SERIAL PRIMARY KEY

Output

1

2

3

4

5

No need to enter the ID manually.

---

3. VARCHAR(n)

Stores variable-length text.

name VARCHAR(20)

✓ Sonu

✓ Rahul

✗ More than 20 characters

---

4. CHAR(n)

Stores fixed-length text.

gender CHAR(1)

✓ M

✓ F

✓ A

---

5. TEXT

Stores large amounts of text.

address TEXT

✓ Paragraphs

✓ Descriptions

✓ Articles

---

6. BOOLEAN

Stores logical values.

active BOOLEAN

✓ TRUE

✓ FALSE

---

7. DATE

Stores only dates.

dob DATE

✓ 2026-07-05

---

8. TIME

Stores only time.

login_time TIME

✓ 10:30:45

---

9. TIMESTAMP

Stores both date and time.

created_at TIMESTAMP

✓ 2026-07-05 10:30:45

---

10. DECIMAL(p,s) / NUMERIC(p,s)

Used for **exact decimal values**, especially money and salaries.

salary DECIMAL(5,2)

Where

- **p** = Total number of digits
- **s** = Number of digits after the decimal point

Example:

DECIMAL(5,2)

Maximum value

999.99

Value	Result
15.35	✓
99.99	✓
123.45	✓
28.1	✓ (Stored as 28.10)
999.99	✓
1000.50	✗
1234.56	✗
15.357	✗

---

11. REAL

Stores approximate decimal numbers (single precision).

height REAL

✓ 15.35

---

12. DOUBLE PRECISION

Stores large decimal numbers with higher precision.

price DOUBLE PRECISION

✓ 1234567.987654

Example Table

```
CREATE TABLE students (
id SERIAL PRIMARY KEY,
name VARCHAR(100),
age INT,
salary DECIMAL(10,2),
```

dob DATE,

active BOOLEAN

);

Difference Between Common Data Types

CHAR vs VARCHAR

CHAR	**VARCHAR**
Fixed length	Variable length
Wastes extra space if data is short	Saves storage space
Example: Gender	Example: Name

INTEGER vs SERIAL

INTEGER	**SERIAL**
Value entered manually	Value generated automatically
Used for Age, Marks	Used for ID

DECIMAL vs REAL

DECIMAL	**REAL**
Exact values	Approximate values
Used for money, salary	Used for scientific calculations

Memory Tricks

Data Type	**Used For**
INT	Whole numbers
SERIAL	Auto-increment IDs

Data Type	Used For
VARCHAR	Variable-length text
CHAR	Fixed-length text
TEXT	Long text
BOOLEAN	TRUE / FALSE
DATE	Date only
TIME	Time only
TIMESTAMP	Date + Time
DECIMAL	Exact decimal values
REAL	Approximate decimal values
DOUBLE PRECISION	High-precision decimal values

Quick Interview / Exam Definition

Data types are attributes that define the type of data a column can store. They help ensure data accuracy, optimize storage, improve performance, and maintain consistency in a database.

PostgreSQL Constraints

What is a Constraint?

A **constraint** is a rule applied to one or more columns in a table to ensure that only **valid, accurate, and consistent data** is stored in the database.

Exam Definition

A constraint is a rule enforced on a table column to restrict the type of data that can be inserted, updated, or deleted, thereby maintaining data integrity.

Why are Constraints Important?

- Ensure data accuracy.
- Prevent invalid or duplicate data.

- Maintain data integrity.
- Improve database reliability.
- Enforce business rules.

Types of Constraints in PostgreSQL

1. PRIMARY KEY
2. NOT NULL
3. UNIQUE
4. DEFAULT
5. CHECK
6. FOREIGN KEY

---

1. PRIMARY KEY

A **Primary Key** uniquely identifies each record in a table.

Properties

- Must contain **unique values**.
- Cannot contain **NULL** values.
- Each table can have **only one Primary Key**.
- Automatically creates a unique index.

Syntax

```
CREATE TABLE students (
id INT PRIMARY KEY,
name VARCHAR(100)
);
```

Example

```
INSERT INTO students VALUES (1,'Sonu');
INSERT INTO students VALUES (2,'Rahul');
```

✓ Valid

INSERT INTO students VALUES (1,'Alex');

❌ Error (Duplicate Primary Key)

INSERT INTO students VALUES (NULL,'Sonu');

❌ Error (NULL not allowed)

---

2. NOT NULL

The **NOT NULL** constraint ensures that a column cannot contain NULL values.

Syntax

CREATE TABLE customers(

id INT NOT NULL,

name VARCHAR(100) NOT NULL

);

Example

✓

INSERT INTO customers VALUES (1,'Sonu');

❌

INSERT INTO customers(id,name)

VALUES (2,NULL);

Error because **name** cannot be NULL.

---

3. UNIQUE

The **UNIQUE** constraint ensures that all values in a column are different.

Syntax

CREATE TABLE employees(

id SERIAL PRIMARY KEY,

email VARCHAR(100) UNIQUE

);

Example

✓

abc@gmail.com

xyz@gmail.com

✗

abc@gmail.com

abc@gmail.com

Duplicate values are not allowed.

Difference Between PRIMARY KEY and UNIQUE

PRIMARY KEY	UNIQUE
Only one per table	Multiple UNIQUE constraints allowed
Cannot contain NULL	Can contain NULL (PostgreSQL allows multiple NULLs)
Must be unique	Must be unique

---

4. DEFAULT

The **DEFAULT** constraint assigns a value automatically if no value is provided.

Syntax

CREATE TABLE customers(

id INT PRIMARY KEY,

name VARCHAR(100),

account_type VARCHAR(20)

DEFAULT 'Savings'

);

Example

INSERT INTO customers(id,name)

VALUES(101,'Sonu');

Output

id	name	account_type
101	Sonu	Savings

Since no value was given, PostgreSQL inserted the **default value**.

---

5. CHECK

The **CHECK** constraint ensures that values satisfy a specified condition.

Syntax

```
CREATE TABLE students(
id SERIAL PRIMARY KEY,
age INT CHECK(age>=18)
);
```

Example

✓

```
INSERT INTO students(age)
VALUES(20);
```

✗

```
INSERT INTO students(age)
VALUES(15);
```

Error because **15 < 18**.

Another Example

```
salary DECIMAL(10,2)
CHECK(salary>0)
```

Salary cannot be negative.

---

6. FOREIGN KEY

A **Foreign Key** creates a relationship between two tables.

It references the **Primary Key** of another table.

Example

Department Table

```
CREATE TABLE departments(
dept_id INT PRIMARY KEY,
dept_name VARCHAR(50)
);
```

Employee Table

```
CREATE TABLE employees(
emp_id SERIAL PRIMARY KEY,
emp_name VARCHAR(50),
dept_id INT,
FOREIGN KEY(dept_id)
REFERENCES departments(dept_id)
);
```

Only department IDs that exist in the **departments** table can be inserted.

AUTO INCREMENT (SERIAL)

PostgreSQL uses **SERIAL** to generate numbers automatically.

Syntax

```
CREATE TABLE employees(
id SERIAL PRIMARY KEY,
firstname VARCHAR(50),
lastname VARCHAR(50)
);
```

Output

1

2

3

4

5

No need to insert the ID manually.

Combined Example

CREATE TABLE students(

id SERIAL PRIMARY KEY,

name VARCHAR(100) NOT NULL,

email VARCHAR(100) UNIQUE,

age INT CHECK(age>=18),

course VARCHAR(50)

DEFAULT 'CSE'

);

Summary Table

Constraint	Purpose	Example
PRIMARY KEY	Uniquely identifies each record	id PRIMARY KEY
NOT NULL	Prevents NULL values	name NOT NULL

Constraint	Purpose	Example
UNIQUE	Prevents duplicate values	email UNIQUE
DEFAULT	Assigns a default value	course DEFAULT 'CSE'
CHECK	Restricts values based on a condition	age CHECK(age>=18)
FOREIGN KEY	Links two tables	dept_id REFERENCES departments(dept_id)
SERIAL	Auto-increment integer	id SERIAL PRIMARY KEY

Quick Interview / Exam Definitions

- **Constraint:** A rule that restricts the data stored in a table to maintain accuracy and integrity.
- **Primary Key:** A column that uniquely identifies each record and cannot contain NULL values.
- **NOT NULL:** Ensures that a column always contains a value.
- **UNIQUE:** Ensures that duplicate values are not allowed in a column.
- **DEFAULT:** Assigns a predefined value when no value is provided.
- **CHECK:** Ensures that data satisfies a specified condition.
- **FOREIGN KEY:** Establishes a relationship between two tables by referencing the primary key of another table.
- **SERIAL:** A PostgreSQL data type that automatically generates sequential integer values, commonly used for primary keys.

Employee Table – Constraints Example (Short Notes)

Objective

Create an **Employee Information** table using different PostgreSQL constraints such as **PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT, SERIAL, and CURRENT_DATE**.

Table Creation

```
CREATE TABLE EMP_INFO(

EMP_ID SERIAL PRIMARY KEY,

FNAME VARCHAR(20) NOT NULL,
```

```
LNAME VARCHAR(15) NOT NULL,
EMAIL VARCHAR(35) NOT NULL UNIQUE,
DEPT VARCHAR(15),
SALARY DECIMAL(10,2) DEFAULT 300000.00,
HIRE_DATE DATE NOT NULL DEFAULT CURRENT_DATE
);
```

Constraints Used

Constraint	Used On	Purpose
SERIAL	EMP_ID	Automatically generates employee IDs.
PRIMARY KEY	EMP_ID	Uniquely identifies each employee.
NOT NULL	FNAME, LNAME, EMAIL, HIRE_DATE	Prevents NULL values.
UNIQUE	EMAIL	Prevents duplicate email addresses.
DEFAULT	SALARY	Sets salary to 300000.00 if not provided.
DEFAULT CURRENT_DATE	HIRE_DATE	Automatically stores today's date if not provided.

Insert Employee Records

```
INSERT INTO emp_info
(emp_id, fname, lname, email, dept, salary, hire_date)
VALUES
(1,'Raj','Sharma','raj.sharma@example.com','IT',50000,'2020-01-15'),
(2,'Priya','Singh','priya.singh@example.com','HR',45000,'2019-03-22'),
...
(10,'Vijay','Nair','vijay.nair@example.com','Marketing',50000,'2020-04-19');
```

View the Data

SELECT * FROM emp_info;

Insert Record Using Default Values

INSERT INTO emp_info

(fname, lname, email, dept)

VALUES

('Sonu','Suman','sonu.suman@example.com','IT');

Output

- **EMP_ID** → Generated automatically by SERIAL.
- **SALARY** → Set to 300000.00 (DEFAULT value).
- **HIRE_DATE** → Set to the current date (CURRENT_DATE).

Check Current Sequence Value

SELECT currval('emp_info_emp_id_seq');

Displays the last generated employee ID.

Reset Sequence

SELECT setval('emp_info_emp_id_seq',10);

Sets the sequence to **10**, so the next inserted employee will receive **EMP_ID = 11**.

Summary

- **PRIMARY KEY** → Unique employee ID.
- **SERIAL** → Auto-generates IDs.
- **NOT NULL** → Mandatory fields.
- **UNIQUE** → No duplicate emails.
- **DEFAULT** → Assigns default salary.

- **CURRENT_DATE** → Automatically stores today's date for new employees.
- **currval()** → Shows the current sequence value.
- **setval()** → Resets or changes the sequence value.

PostgreSQL Clauses

What is a Clause?

A **clause** is a part of an SQL statement that performs a specific task, such as filtering, sorting, or limiting data.

Exam Definition

A clause is a component of an SQL query used to filter, sort, group, or limit the data returned by the database.

---

1. WHERE Clause

The **WHERE** clause is used to retrieve records that satisfy a specified condition.

Syntax

SELECT * FROM table_name

WHERE condition;

Example

SELECT * FROM emp_info

WHERE dept = 'IT';

---

2. AND / OR Operators

Used to combine multiple conditions.

AND

Returns records only if **all conditions** are true.

SELECT * FROM emp_info

WHERE dept='IT' AND salary>50000;

OR

Returns records if **any one condition** is true.

SELECT * FROM emp_info

WHERE dept='HR' OR dept='Finance';

---

3. IN Operator

Used to match multiple values.

Syntax

SELECT * FROM emp_info

WHERE dept IN ('IT','HR');

Equivalent to

WHERE dept='IT' OR dept='HR'

---

4. NOT IN Operator

Returns records **not matching** the given values.

SELECT * FROM emp_info

WHERE dept NOT IN ('IT','HR');

---

5. BETWEEN Operator

Used to retrieve values within a range.

Syntax

SELECT * FROM emp_info

WHERE salary BETWEEN 45000 AND 55000;

Date Example

SELECT * FROM emp_info

WHERE hire_date BETWEEN '2020-01-01' AND '2021-12-31';

---

6. DISTINCT Clause

Removes duplicate values.

Syntax

SELECT DISTINCT dept

FROM emp_info;

Output

Finance

HR

IT

Marketing

---

7. ORDER BY Clause

Sorts records in ascending or descending order.

Ascending (Default)

SELECT * FROM emp_info

ORDER BY salary;

or

SELECT * FROM emp_info

ORDER BY salary ASC;

Descending

SELECT * FROM emp_info

ORDER BY salary DESC;

Sort by Multiple Columns

SELECT * FROM emp_info

ORDER BY dept ASC, salary DESC;

---

8. LIMIT Clause

Limits the number of rows returned.

Syntax

SELECT * FROM emp_info

LIMIT 5;

First 5 records only.

Example

SELECT * FROM emp_info

ORDER BY salary DESC

LIMIT 3;

Returns the top 3 highest-paid employees.

---

9. LIKE Operator

Used for pattern matching.

1. Starts With

SELECT * FROM emp_info

WHERE fname LIKE 'R%';

Names starting with **R**

Example

Raj

Rahul

---

2. Ends With

SELECT * FROM emp_info

WHERE fname LIKE '%a';

Names ending with **a**

Example

Priya

Neha

---

3. Contains

SELECT * FROM emp_info

WHERE fname LIKE '%aj%';

Contains **aj**

Example

Raj

---

4. Second Character

SELECT * FROM emp_info

WHERE fname LIKE '_a%';

Second character is **a**

Example

Raj

Kavita

Rahul

_ represents exactly **one character**.

---

5. Exactly Four Characters

SELECT * FROM emp_info

WHERE fname LIKE '____';

Example

Neha

Amit

Each _ represents one character.

---

10. ILIKE (Case-Insensitive Search)

LIKE is **case-sensitive** in PostgreSQL, while ILIKE is **case-insensitive**.

LIKE

SELECT * FROM emp_info

WHERE fname LIKE 'raj%';

Matches only lowercase raj...

ILIKE

SELECT * FROM emp_info

WHERE fname ILIKE 'raj%';

Matches

Raj

RAJ

raj

rAj

---

11. Comments

Single-Line Comment

-- Display all employees

SELECT * FROM emp_info;

Multi-Line Comment

/*

This query displays

employees from the IT department

*/

SELECT * FROM emp_info

WHERE dept='IT';

Summary Table

Clause / Operator	Purpose	Example
WHERE	Filter records	WHERE dept='IT'
AND	All conditions must be true	dept='IT' AND salary>50000
OR	Any condition can be true	dept='HR' OR dept='IT'
IN	Match multiple values	dept IN('IT','HR')
NOT IN	Exclude multiple values	dept NOT IN('HR','IT')
BETWEEN	Values within a range	salary BETWEEN 40000 AND 60000
DISTINCT	Remove duplicate values	SELECT DISTINCT dept
ORDER BY	Sort data	ORDER BY salary DESC
LIMIT	Limit returned rows	LIMIT 5
LIKE	Pattern matching	LIKE 'R%'
ILIKE	Case-insensitive pattern matching	ILIKE 'raj%'
--	Single-line comment	-- comment
/* */	Multi-line comment	/* comment */

Most Common LIKE Patterns

Pattern	Meaning	Example
'R%'	Starts with **R**	Raj, Rahul
'%a'	Ends with **a**	Priya, Neha
'%aj%'	Contains **aj**	Raj
'_a%'	Second character is **a**	Raj, Rahul
'____'	Exactly 4 characters	Neha, Amit

Quick Interview Definition

SQL clauses are components of an SQL query that perform specific operations such as filtering (WHERE), sorting (ORDER BY), removing duplicates (DISTINCT), limiting results (LIMIT), and pattern matching (LIKE).

PostgreSQL Aggregate Functions

What are Aggregate Functions?

Aggregate functions perform calculations on multiple rows and return **a single result**. They are commonly used for data analysis, reports, and summaries.

Exam Definition

Aggregate functions are SQL functions that perform calculations on a set of values and return a single summarized result.

Why are Aggregate Functions Used?

- Calculate totals.
- Find maximum and minimum values.
- Count records.
- Calculate averages.
- Generate summary reports.

Common Aggregate Functions

Function	Purpose	Example
COUNT()	Counts rows	COUNT(*)
SUM()	Calculates total	SUM(salary)
AVG()	Calculates average	AVG(salary)
MAX()	Finds maximum value	MAX(salary)
MIN()	Finds minimum value	MIN(salary)

Note: There is **no SUB() aggregate function** in SQL. If you meant subtraction, SQL uses the - operator, not an aggregate function.

Sample Table

Assume the table **emp_info** contains:

EMP_ID	FNAME	DEPT	SALARY
1	Raj	IT	50000
2	Priya	HR	45000
3	Arjun	IT	55000
4	Suman	Finance	60000
5	Kavita	HR	47000

---

1. COUNT()

Counts the number of records.

Syntax

SELECT COUNT(*) FROM emp_info;

Output

5

Count Non-NULL Values

SELECT COUNT(email)

FROM emp_info;

Count Employees in IT

SELECT COUNT(*)

FROM emp_info

WHERE dept='IT';

---

2. SUM()

Calculates the total of numeric values.

Syntax

```
SELECT SUM(salary)
FROM emp_info;
```

Output

257000

Total Salary of IT Department

```
SELECT SUM(salary)
FROM emp_info
WHERE dept='IT';
```

---

3. AVG()

Calculates the average value.

Syntax

```
SELECT AVG(salary)
FROM emp_info;
```

Average Salary of HR

```
SELECT AVG(salary)
FROM emp_info
WHERE dept='HR';
```

---

4. MAX()

Returns the highest value.

Syntax

```
SELECT MAX(salary)
FROM emp_info;
```

Output

60000

Highest Salary in IT

```
SELECT MAX(salary)
FROM emp_info
WHERE dept='IT';
```

---

5. MIN()

Returns the smallest value.

Syntax

```
SELECT MIN(salary)
FROM emp_info;
```

Output

45000

Lowest Salary in HR

```
SELECT MIN(salary)
FROM emp_info
WHERE dept='HR';
```

---

6. Aggregate Functions with WHERE

```
SELECT SUM(salary)
FROM emp_info
WHERE dept='Finance';
```

Only Finance employees are considered.

---

7. Aggregate Functions with DISTINCT

Count Different Departments

```
SELECT COUNT(DISTINCT dept)
FROM emp_info;
```

Total of Unique Salaries

```
SELECT SUM(DISTINCT salary)
FROM emp_info;
```

---

8. Aggregate Functions with Aliases

```
SELECT
MAX(salary) AS Highest_Salary,
MIN(salary) AS Lowest_Salary,
AVG(salary) AS Average_Salary
FROM emp_info;
```

---

9. GROUP BY

Groups records having the same value.

Syntax

```
SELECT dept,
COUNT(*)
FROM emp_info
GROUP BY dept;
```

Output

Department	Employees
IT	2
HR	2
Finance	1

Average Salary by Department

```
SELECT dept,
AVG(salary)
FROM emp_info
GROUP BY dept;
```

10. HAVING

Filters grouped data.

Syntax

SELECT dept,

COUNT(*)

FROM emp_info

GROUP BY dept

HAVING COUNT(*) > 1;

Shows only departments having more than one employee.

Multiple Aggregate Functions Together

SELECT

COUNT(*) AS Total_Employees,

SUM(salary) AS Total_Salary,

AVG(salary) AS Average_Salary,

MAX(salary) AS Highest_Salary,

MIN(salary) AS Lowest_Salary

FROM emp_info;

Difference Between WHERE and HAVING

WHERE	**HAVING**
Filters rows before grouping	Filters groups after grouping
Used before GROUP BY	Used after GROUP BY
Cannot use aggregate functions directly	Can use aggregate functions

Example using **WHERE**

SELECT *

```
FROM emp_info

WHERE salary > 50000;
```

Example using **HAVING**

```
SELECT dept,

AVG(salary)

FROM emp_info

GROUP BY dept

HAVING AVG(salary) > 50000;
```

Summary Table

Function	Purpose	Example
COUNT()	Counts records	COUNT(*)
SUM()	Calculates total	SUM(salary)
AVG()	Calculates average	AVG(salary)
MAX()	Finds maximum value	MAX(salary)
MIN()	Finds minimum value	MIN(salary)
GROUP BY	Groups rows	GROUP BY dept
HAVING	Filters grouped results	HAVING COUNT(*) > 2

Quick Interview / Exam Definitions

- **COUNT()** – Counts the number of rows or non-NULL values.
- **SUM()** – Returns the total of numeric values.
- **AVG()** – Returns the average of numeric values.
- **MAX()** – Returns the highest value.
- **MIN()** – Returns the lowest value.
- **GROUP BY** – Groups rows with the same values into summary rows.

- **HAVING** – Filters grouped data based on aggregate conditions.

Note: AVG(), GROUP BY, and HAVING are also essential topics when studying aggregate functions, so they are typically included together in PostgreSQL and SQL coursework.

PostgreSQL String Functions

What are String Functions?

String functions are SQL functions used to manipulate, modify, search, and format text (character) data.

Exam Definition

String functions are built-in SQL functions used to perform operations on character or text data, such as joining, extracting, replacing, converting case, and trimming spaces.

Why are String Functions Used?

- Combine text values.
- Extract characters from strings.
- Convert text to uppercase or lowercase.
- Remove extra spaces.
- Replace words or characters.
- Find the position of a character or substring.

Common PostgreSQL String Functions

Function	Purpose
CONCAT()	Combines two or more strings
CONCAT_WS()	Combines strings with a separator
SUBSTR() / SUBSTRING()	Extracts part of a string
LEFT()	Returns leftmost characters
RIGHT()	Returns rightmost characters
LENGTH()	Returns the length of a string
UPPER()	Converts text to uppercase

Function	Purpose
LOWER()	Converts text to lowercase
TRIM()	Removes spaces from both sides
LTRIM()	Removes spaces from the left side
RTRIM()	Removes spaces from the right side
REPLACE()	Replaces part of a string
POSITION()	Finds the position of a substring

Sample Table

Assume the table **emp_info** contains:

EMP_ID	FNAME	LNAME
1	Raj	Sharma
2	Priya	Singh
3	Arjun	Verma

---

1. CONCAT()

Combines two or more strings.

Syntax

SELECT CONCAT(string1,string2,...);

Example

SELECT CONCAT(fname,' ',lname)

FROM emp_info;

Output

Raj Sharma

Priya Singh

Arjun Verma

2. CONCAT_WS()

WS = With Separator

Combines strings using a separator.

Syntax

SELECT CONCAT_WS('-',fname,lname);

Output

Raj-Sharma

Priya-Singh

Arjun-Verma

Another Example

SELECT CONCAT_WS(', ','Raj','IT','50000');

Output

Raj, IT, 50000

---

3. SUBSTR() / SUBSTRING()

Extracts a part of a string.

Syntax

SELECT SUBSTR('PostgreSQL',1,4);

Output

Post

Employee Example

SELECT SUBSTR(fname,1,3)

FROM emp_info;

Output

Raj

Pri

Arj

---

4. LEFT()

Returns characters from the left.

Syntax

SELECT LEFT('PostgreSQL',4);

Output

Post

Example

SELECT LEFT(fname,2)

FROM emp_info;

Output

Ra

Pr

Ar

---

5. RIGHT()

Returns characters from the right.

Syntax

SELECT RIGHT('PostgreSQL',3);

Output

SQL

Example

SELECT RIGHT(fname,2)

FROM emp_info;

---

6. LENGTH()

Returns the number of characters.

Syntax

SELECT LENGTH('PostgreSQL');

Output

10

Example

SELECT fname,

LENGTH(fname)

FROM emp_info;

Output

Raj 3

Priya 5

Arjun 5

---

7. UPPER()

Converts text into uppercase.

Syntax

SELECT UPPER(fname)

FROM emp_info;

Output

RAJ

PRIYA

ARJUN

---

8. LOWER()

Converts text into lowercase.

Syntax

```
SELECT LOWER(fname)
FROM emp_info;
```

Output

raj

priya

arjun

---

9. TRIM()

Removes spaces from both ends.

Syntax

```
SELECT TRIM('   PostgreSQL   ');
```

Output

PostgreSQL

---

10. LTRIM()

Removes spaces from the left.

Syntax

```
SELECT LTRIM('   PostgreSQL');
```

Output

PostgreSQL

---

11. RTRIM()

Removes spaces from the right.

Syntax

```
SELECT RTRIM('PostgreSQL   ');
```

Output

PostgreSQL

---

12. REPLACE()

Replaces part of a string.

Syntax

SELECT REPLACE('PostgreSQL','SQL','Database');

Output

PostgreDatabase

Example

SELECT REPLACE(email,'@example.com','@gmail.com')

FROM emp_info;

---

13. POSITION()

Returns the position of a substring.

Syntax

SELECT POSITION('SQL' IN 'PostgreSQL');

Output

8

Example

SELECT POSITION('@' IN email)

FROM emp_info;

Returns the position of **@** in every email.

Nested String Functions

Functions can be combined.

Example

SELECT UPPER(CONCAT(fname,' ',lname))

FROM emp_info;

Output

RAJ SHARMA

PRIYA SINGH

ARJUN VERMA

Summary Table

Function	Purpose	Example
CONCAT()	Combines strings	CONCAT(fname,lname)
CONCAT_WS()	Combines strings with separator	CONCAT_WS('-',fname,lname)
SUBSTR()	Extracts part of a string	SUBSTR(fname,1,3)
LEFT()	Leftmost characters	LEFT(fname,2)
RIGHT()	Rightmost characters	RIGHT(fname,2)
LENGTH()	Number of characters	LENGTH(fname)
UPPER()	Converts to uppercase	UPPER(fname)
LOWER()	Converts to lowercase	LOWER(fname)
TRIM()	Removes spaces from both ends	TRIM(name)
LTRIM()	Removes left spaces	LTRIM(name)
RTRIM()	Removes right spaces	RTRIM(name)
REPLACE()	Replaces text	REPLACE(email,'@example.com','@gmail.com')
POSITION()	Finds position of substring	POSITION('@' IN email)

Quick Interview / Exam Definitions

- **CONCAT()** – Combines two or more strings into one.
- **CONCAT_WS()** – Combines strings using a specified separator.
- **SUBSTR()** – Extracts a specified part of a string.
- **LEFT()** – Returns the leftmost characters of a string.

- **RIGHT()** – Returns the rightmost characters of a string.
- **LENGTH()** – Returns the total number of characters in a string.
- **UPPER()** – Converts all characters to uppercase.
- **LOWER()** – Converts all characters to lowercase.
- **TRIM()** – Removes spaces from both the beginning and end of a string.
- **LTRIM()** – Removes leading (left) spaces.
- **RTRIM()** – Removes trailing (right) spaces.
- **REPLACE()** – Replaces occurrences of a substring with another substring.
- **POSITION()** – Returns the starting position of a specified substring within a string.

PostgreSQL ALTER TABLE

What is ALTER TABLE?

The **ALTER TABLE** statement is used to modify the structure of an existing table without deleting its data.

Exam Definition

ALTER TABLE is an SQL command used to add, delete, rename, or modify columns and other properties of an existing table.

Why is ALTER TABLE Used?

- Add new columns.
- Remove existing columns.
- Rename columns.
- Rename tables.
- Change a column's data type.
- Add or remove default values.
- Add or remove constraints.

Syntax

ALTER TABLE table_name

action;

---

1. Add a New Column

Syntax

ALTER TABLE table_name

ADD COLUMN column_name datatype;

Example

ALTER TABLE emp_info

ADD COLUMN mobile VARCHAR(15);

Add Multiple Columns

ALTER TABLE emp_info

ADD COLUMN address TEXT,

ADD COLUMN city VARCHAR(30);

---

2. Drop (Remove) a Column

Syntax

ALTER TABLE table_name

DROP COLUMN column_name;

Example

ALTER TABLE emp_info

DROP COLUMN mobile;

---

3. Rename a Column

Syntax

ALTER TABLE table_name

RENAME COLUMN old_column TO new_column;

Example

ALTER TABLE emp_info

RENAME COLUMN fname TO first_name;

---

4. Rename a Table

Syntax

ALTER TABLE old_table_name

RENAME TO new_table_name;

Example

ALTER TABLE emp_info

RENAME TO employee_info;

---

5. Change (Modify) a Column Data Type

Syntax

ALTER TABLE table_name

ALTER COLUMN column_name

TYPE new_datatype;

Example

ALTER TABLE emp_info

ALTER COLUMN salary

TYPE BIGINT;

Another Example

ALTER TABLE emp_info

ALTER COLUMN fname

TYPE VARCHAR(50);

---

6. Add a DEFAULT Value

Syntax

ALTER TABLE table_name

ALTER COLUMN column_name

SET DEFAULT value;

Example

ALTER TABLE emp_info

ALTER COLUMN dept

SET DEFAULT 'IT';

Now, if no department is entered, PostgreSQL automatically stores **IT**.

---

7. Remove DEFAULT Value

Syntax

ALTER TABLE table_name

ALTER COLUMN column_name

DROP DEFAULT;

Example

ALTER TABLE emp_info

ALTER COLUMN dept

DROP DEFAULT;

---

8. Set NOT NULL Constraint

Syntax

ALTER TABLE table_name

ALTER COLUMN column_name

SET NOT NULL;

Example

ALTER TABLE emp_info

```
ALTER COLUMN email
SET NOT NULL;
```

---

9. Remove NOT NULL Constraint

Syntax

```
ALTER TABLE table_name
ALTER COLUMN column_name
DROP NOT NULL;
```

Example

```
ALTER TABLE emp_info
ALTER COLUMN dept
DROP NOT NULL;
```

---

10. Add UNIQUE Constraint

Syntax

```
ALTER TABLE table_name
ADD CONSTRAINT constraint_name
UNIQUE(column_name);
```

Example

```
ALTER TABLE emp_info
ADD CONSTRAINT unique_email
UNIQUE(email);
```

---

11. Drop UNIQUE Constraint

Syntax

```
ALTER TABLE table_name
DROP CONSTRAINT constraint_name;
```

Example

ALTER TABLE emp_info

DROP CONSTRAINT unique_email;

---

12. Add PRIMARY KEY

Syntax

ALTER TABLE table_name

ADD PRIMARY KEY(column_name);

Example

ALTER TABLE emp_info

ADD PRIMARY KEY(emp_id);

---

13. Drop PRIMARY KEY

Syntax

ALTER TABLE table_name

DROP CONSTRAINT emp_info_pkey;

Note: PostgreSQL stores the primary key as a constraint. The default name is usually table_name_pkey.

View Table Structure

\d emp_info

Displays all columns, data types, and constraints.

Summary Table

Operation	Syntax
Add Column	ADD COLUMN column_name datatype
Drop Column	DROP COLUMN column_name

Operation	Syntax
Rename Column	RENAME COLUMN old TO new
Rename Table	RENAME TO new_table
Change Data Type	ALTER COLUMN column TYPE datatype
Add Default	ALTER COLUMN column SET DEFAULT value
Remove Default	ALTER COLUMN column DROP DEFAULT
Set NOT NULL	ALTER COLUMN column SET NOT NULL
Remove NOT NULL	ALTER COLUMN column DROP NOT NULL
Add UNIQUE	ADD CONSTRAINT name UNIQUE(column)
Drop UNIQUE	DROP CONSTRAINT constraint_name
Add PRIMARY KEY	ADD PRIMARY KEY(column)
Drop PRIMARY KEY	DROP CONSTRAINT table_name_pkey

Complete Example

```
-- Add a column
ALTER TABLE emp_info
ADD COLUMN phone VARCHAR(15);


-- Rename the column
ALTER TABLE emp_info
RENAME COLUMN phone TO mobile;


-- Change the data type
ALTER TABLE emp_info
ALTER COLUMN mobile TYPE VARCHAR(20);
```

```
-- Add a default value
ALTER TABLE emp_info
ALTER COLUMN dept SET DEFAULT 'IT';

-- Remove the default value
ALTER TABLE emp_info
ALTER COLUMN dept DROP DEFAULT;

-- Drop the column
ALTER TABLE emp_info
DROP COLUMN mobile;
```

Quick Interview / Exam Definitions

- **ALTER TABLE:** Modifies the structure of an existing table.
- **ADD COLUMN:** Adds a new column to a table.
- **DROP COLUMN:** Removes a column from a table.
- **RENAME COLUMN:** Changes the name of an existing column.
- **RENAME TO:** Changes the name of a table.
- **ALTER COLUMN TYPE:** Changes the data type of a column.
- **SET DEFAULT:** Assigns a default value to a column.
- **DROP DEFAULT:** Removes the default value from a column.
- **SET NOT NULL:** Makes a column mandatory.
- **DROP NOT NULL:** Allows NULL values in a column.

Named Constraints in PostgreSQL (Short Notes – A4 Size, ~2 Pages)

What is a Named Constraint?

A **Named Constraint** is a constraint that is assigned a **custom name** by the user while creating or modifying a table. Naming constraints makes them easier to identify, manage, and remove later.

Exam Definition

A Named Constraint is a user-defined name given to a database constraint to improve readability and simplify maintenance.

Why Use Named Constraints?

- Easy to identify constraints.
- Easy to modify or delete.
- Improves code readability.
- Simplifies database maintenance.
- Useful in large database applications.

Syntax

CONSTRAINT constraint_name

constraint_type(column_name)

General Syntax

CREATE TABLE table_name(

column_name datatype,

CONSTRAINT constraint_name

constraint_type(column_name)

);

Types of Named Constraints

- PRIMARY KEY
- UNIQUE
- CHECK
- FOREIGN KEY

Note: NOT NULL and DEFAULT are column properties and are generally not named in PostgreSQL.

---

1. Named PRIMARY KEY

A **PRIMARY KEY** uniquely identifies each record in a table.

```
CREATE TABLE employee(
emp_id INT,
CONSTRAINT pk_employee
PRIMARY KEY(emp_id)
);
```

Purpose: Ensures emp_id is unique and cannot be NULL.

---

2. Named UNIQUE

A **UNIQUE** constraint prevents duplicate values.

```
CREATE TABLE employee(
email VARCHAR(50),
CONSTRAINT uq_employee_email
UNIQUE(email)
);
```

Purpose: Ensures each employee has a unique email address.

---

3. Named CHECK

A **CHECK** constraint validates data before insertion.

```
CREATE TABLE employee(
salary DECIMAL(10,2),
CONSTRAINT chk_salary
CHECK(salary > 0)
);
```

Purpose: Salary must always be greater than zero.

---

4. Named FOREIGN KEY

A **FOREIGN KEY** creates a relationship between two tables.

```
CREATE TABLE department(
dept_id INT PRIMARY KEY
);

CREATE TABLE employee(
emp_id INT PRIMARY KEY,
dept_id INT,
CONSTRAINT fk_department
FOREIGN KEY(dept_id)
REFERENCES department(dept_id)
);
```

Purpose: Allows only valid department IDs.

Adding a Named Constraint

```
ALTER TABLE employee
ADD CONSTRAINT uq_email
UNIQUE(email);
```

Dropping a Named Constraint

```
ALTER TABLE employee
DROP CONSTRAINT uq_email;
```

Renaming a Constraint

```
ALTER TABLE employee
RENAME CONSTRAINT uq_email
```

TO unique_email;

View Constraints

\d employee

or

SELECT *

FROM information_schema.table_constraints

WHERE table_name='employee';

Naming Convention

Constraint	Prefix	Example
PRIMARY KEY	pk_	pk_employee
UNIQUE	uq_	uq_employee_email
CHECK	chk_	chk_salary
FOREIGN KEY	fk_	fk_department

Named vs Unnamed Constraints

Named Constraint	Unnamed Constraint
User gives the name	PostgreSQL generates the name
Easy to identify	Hard to identify
Easy to drop or modify	Must find generated name first

Advantages

- Easy to manage.
- Improves readability.
- Easy to modify or remove.

- Helps in debugging.
- Recommended for professional database design.

Complete Example

```
CREATE TABLE employee(
emp_id SERIAL,
fname VARCHAR(30) NOT NULL,
email VARCHAR(50),
salary DECIMAL(10,2),
dept_id INT,
CONSTRAINT pk_employee
PRIMARY KEY(emp_id),
CONSTRAINT uq_employee_email
UNIQUE(email),
CONSTRAINT chk_salary
CHECK(salary > 0),
CONSTRAINT fk_department
FOREIGN KEY(dept_id)
REFERENCES department(dept_id)
);
```

Summary

- **Named Constraints** are user-defined names given to database constraints.
- They improve readability, maintenance, and debugging.
- Common named constraints are:
    - **PRIMARY KEY**
    - **UNIQUE**

- **CHECK**
- **FOREIGN KEY**

- They can be **added**, **renamed**, and **dropped** using the ALTER TABLE statement.

Interview Definition

A Named Constraint is a user-defined identifier assigned to a database constraint, making it easier to identify, manage, modify, and remove constraints in a database.

CASE Statement in PostgreSQL – Complete Notes

What is the CASE Statement?

The **CASE** statement is a conditional expression in SQL. It works like the **if-else statement** in programming languages. It checks one or more conditions and returns a value based on the first condition that evaluates to **TRUE**.

Exam Definition

The CASE statement is used to apply conditional logic in SQL queries. It evaluates conditions and returns different values depending on which condition is satisfied.

Why Use CASE?

- Categorize data.
- Display custom values.
- Calculate bonuses or grades.
- Replace complex IF-ELSE logic.
- Use conditions inside SELECT statements.

Types of CASE

1. Simple CASE

Compares one expression with multiple values.

CASE expression

WHEN value1 THEN result1

WHEN value2 THEN result2

ELSE result

END

---

2. Searched CASE (Most Common)

Checks multiple conditions.

CASE

WHEN condition1 THEN result1

WHEN condition2 THEN result2

ELSE result

END

Syntax

SELECT column_name,

CASE

WHEN condition THEN value

WHEN condition THEN value

ELSE value

END AS alias_name

FROM table_name;

EMP_INFO Table

The following examples use the **EMP_INFO** table.

EMP_ID	FNAME	DEPT	SALARY
1	Raj	IT	50000

EMP_ID	FNAME	DEPT	SALARY
2	Priya	HR	45000
3	Arjun	IT	55000
4	Suman	Finance	60000
5	Kavita	HR	47000
6	Amit	Marketing	52000
7	Neha	IT	48000
8	Rahul	IT	53000
9	Anjali	Finance	61000
10	Vijay	Marketing	50000
11	Sonu	IT	300000

Example 1 – Salary Category

```
SELECT fname,salary,
CASE
WHEN salary>=50000
THEN 'HIGH SALARY'
WHEN salary>=40000
THEN 'MID SALARY'
ELSE 'LOW SALARY'
END AS sal_cat
FROM emp_info
ORDER BY fname;
```

Output

Employee	Salary	Category
Amit	52000	HIGH SALARY
Anjali	61000	HIGH SALARY
Arjun	55000	HIGH SALARY
Kavita	47000	MID SALARY
Neha	48000	MID SALARY
Priya	45000	MID SALARY
Rahul	53000	HIGH SALARY
Raj	50000	HIGH SALARY
Sonu	300000	HIGH SALARY
Suman	60000	HIGH SALARY
Vijay	50000	HIGH SALARY

Explanation

- Salary **50000 or more** → HIGH SALARY
- Salary **40000–49999** → MID SALARY
- Otherwise → LOW SALARY

Example 2 – Salary Category Using BETWEEN

```
SELECT fname,salary,
CASE


WHEN salary>=55000
THEN 'HIGH'
WHEN salary BETWEEN 50000 AND 55000
THEN 'MID'
ELSE 'LOW SALARY'
```

```
END AS sal_cat
FROM emp_info
ORDER BY fname;
```

Explanation

- Salary **55000 or above** → HIGH
- Salary **50000–55000** → MID
- Remaining salaries → LOW SALARY

Note: CASE checks conditions from top to bottom. Since salary >= 55000 is checked first, a salary of **55000** will be classified as **HIGH**, not **MID**.

Example 3 – Calculate Bonus

```
SELECT fname,
salary,
CASE
WHEN salary>0
THEN ROUND(salary*0.10)
END AS bonus
FROM emp_info;
```

Explanation

- Every employee receives **10% bonus**.
- ROUND() rounds the bonus to the nearest whole number.

Sample Output

Employee	Salary	Bonus
Raj	50000	5000
Priya	45000	4500
Sonu	300000	30000

Example 4 – Count Employees by Salary Category

```
SELECT
CASE
WHEN salary>55000 THEN 'HIGH'
WHEN salary BETWEEN 50000 AND 55000
THEN 'MID'
ELSE 'LOW'
END AS sal_cat,
COUNT(emp_id)
FROM emp_info
GROUP BY sal_cat;
```

Sample Output

Salary Category	Employees
HIGH	3
MID	5
LOW	3

(Based on the current EMP_INFO data.)

Explanation

- Employees are first categorized.
- COUNT() counts the employees in each category.
- GROUP BY groups employees with the same category.

CASE with Aggregate Functions

Count Employees

```
SELECT
CASE
WHEN dept='IT'
```

```
THEN 'IT Department'
ELSE 'Other Department'
END,
COUNT(*)
FROM emp_info
GROUP BY 1;
```

Sum of Salary

```
SELECT
CASE
WHEN dept='IT'
THEN 'IT'
ELSE 'Other'
END,
SUM(salary)
FROM emp_info
GROUP BY 1;
```

Average Salary

```
SELECT
CASE
WHEN salary>=50000
THEN 'HIGH'

ELSE 'LOW'
END,
AVG(salary)
```

```
FROM emp_info

GROUP BY 1;
```

Important Points

- CASE works like an **IF-ELSE** statement.
- Conditions are checked **from top to bottom**.
- The **first TRUE condition** is executed.
- If no condition is TRUE, the **ELSE** block is executed.
- ELSE is optional. If omitted and no condition matches, SQL returns **NULL**.
- CASE can be used with:
    - SELECT
    - ORDER BY
    - GROUP BY
    - HAVING
    - Aggregate functions (COUNT, SUM, AVG, MAX, MIN)

Difference Between CASE and WHERE

CASE	WHERE
Returns different values based on conditions	Filters rows based on conditions
Used in SELECT	Used after FROM
Does not remove rows	Removes rows that do not match

Summary

- **CASE** is used to perform conditional logic in SQL.
- It behaves like an **IF-ELSE** statement.
- It helps classify, calculate, and display custom values.
- It can be combined with aggregate functions such as **COUNT()**, **SUM()**, **AVG()**, **MAX()**, and **MIN()**.

- CASE is widely used in reports, dashboards, salary calculations, grading systems, and data analysis.

Interview Definition

The CASE statement is a conditional SQL expression that evaluates one or more conditions and returns a value based on the first matching condition. It is commonly used to categorize data, calculate values, and implement IF-ELSE logic within SQL queries.

PostgreSQL Relationships & Joins

What is a Relationship?

A **relationship** is a connection between two or more tables in a database. Relationships help organize data and reduce duplication.

Exam Definition

A relationship is an association between tables based on a common column (usually a Primary Key and a Foreign Key).

Types of Relationships

There are **3 types** of relationships:

1. One-to-One (1:1)
2. One-to-Many (1:M)
3. Many-to-Many (M:N)

---

1. One-to-One (1:1)

In a **One-to-One** relationship, **one record in the first table is related to only one record in the second table**, and vice versa.

Example

Person	Passport
Raj	P101
Priya	P102

One person has **one passport**.

Person

1 -------- 1

Passport

---

2. One-to-Many (1:M)

In a **One-to-Many** relationship, **one record in the first table can have many related records in the second table**, but each record in the second table belongs to only one record in the first table.

Example

Customers Table

Cust_ID	Name
1	Raju
2	Sham
3	Paul

Orders Table

Order_ID	Cust_ID
101	1
102	1
103	2

Customer **Raju** has **2 orders**.

Customers

|

| 1

|

|------< Many

Orders

This is the relationship used in your example.

---

3. Many-to-Many (M:N)

In a **Many-to-Many** relationship, **many records in one table can relate to many records in another table**.

Example

Students and Courses

Student	**Course**
Raj	Java
Raj	Python
Paul	Java
Paul	SQL

Students >------< Courses

This relationship is usually implemented using a **junction (bridge) table**.

Customer and Orders Tables

Customers Table

```
CREATE TABLE customers(
cust_id SERIAL PRIMARY KEY,
cust_name VARCHAR(100) NOT NULL
);
```

Orders Table

```
CREATE TABLE orders(
ord_id SERIAL PRIMARY KEY,
ord_date DATE NOT NULL,
price NUMERIC NOT NULL,
cust_id INTEGER NOT NULL,
FOREIGN KEY(cust_id)
REFERENCES customers(cust_id)
);
```

Here,

- customers.cust_id → Primary Key
- orders.cust_id → Foreign Key

This creates a **One-to-Many Relationship**.

What is a JOIN?

A **JOIN** is used to combine data from **two or more tables** based on a related column.

Exam Definition

A JOIN combines rows from two or more tables using a related column, usually a Primary Key and Foreign Key.

Types of Joins

1. CROSS JOIN
2. INNER JOIN
3. LEFT JOIN
4. RIGHT JOIN
5. FULL OUTER JOIN *(optional but important)*

Sample Data

Customers

ID	Name
1	Raju
2	Sham
3	Paul
4	Alex

Orders

Order Customer

101 1

102 1

103 2

104 3

105 2

Alex has **no orders**.

---

1. CROSS JOIN

Returns **every row from the first table with every row from the second table**.

Syntax

SELECT *

FROM customers

CROSS JOIN orders;

If

Customers = **4**

Orders = **5**

Result = **20 rows**

4 × 5 = 20

Use when every combination is needed.

---

2. INNER JOIN

Returns **only matching records** from both tables.

Syntax

SELECT *

FROM customers c

INNER JOIN orders o

ON c.cust_id=o.cust_id;

Output

Customer	Order
Raju	101
Raju	102
Sham	103
Sham	105
Paul	104

Alex is **not displayed** because he has no orders.

---

3. LEFT JOIN

Returns **all records from the left table** and matching records from the right table.

Syntax

SELECT *

FROM customers c

LEFT JOIN orders o

ON c.cust_id=o.cust_id;

Output

Customer	Order
Raju	101
Raju	102
Sham	103
Sham	105
Paul	104
Alex	NULL

Alex appears even though he has no orders.

---

4. RIGHT JOIN

Returns **all records from the right table** and matching records from the left table.

Syntax

SELECT *

FROM customers c

RIGHT JOIN orders o

ON c.cust_id=o.cust_id;

Output

All orders are displayed.

If an order has no matching customer, customer columns become **NULL**.

---

5. FULL OUTER JOIN

Returns **all rows from both tables**.

Matching rows are combined.

Non-matching rows contain **NULL** values.

Syntax

SELECT *

FROM customers c

FULL OUTER JOIN orders o

ON c.cust_id=o.cust_id;

Difference Between Joins

Join	Returns
CROSS JOIN	Every possible combination
INNER JOIN	Only matching rows

Join	Returns
LEFT JOIN	All rows from left + matching rows
RIGHT JOIN	All rows from right + matching rows
FULL OUTER JOIN	All rows from both tables

Join Diagram

INNER JOIN

Customers ∩ Orders

Only common records.

LEFT JOIN

Customers + Matching Orders

All customers are shown.

RIGHT JOIN

Matching Customers + All Orders

All orders are shown.

FULL OUTER JOIN

Customers + Orders

Everything is shown.

Summary

- A **relationship** connects tables using **Primary Key** and **Foreign Key**.
- Your example uses a **One-to-Many** relationship.
- A **JOIN** combines data from multiple tables.
- **INNER JOIN** → Matching rows only.

- **LEFT JOIN** → All left table rows.
- **RIGHT JOIN** → All right table rows.
- **CROSS JOIN** → Every possible combination.
- **FULL OUTER JOIN** → All rows from both tables.

Interview / Exam Definitions

Relationship

A relationship is a connection between two tables using a **Primary Key** and a **Foreign Key**.

Join

A JOIN is an SQL operation used to retrieve related data from two or more tables based on a common column.

One-to-One

One record in one table is related to only one record in another table.

One-to-Many

One record in one table can be related to many records in another table.

Many-to-Many

Many records in one table can be related to many records in another table through a junction table.

TASK

Shopping Store Relationships (One-to-Many & Many-to-Many) – Notes

What is a Relationship?

A **relationship** connects two or more tables in a database using **Primary Keys** and **Foreign Keys**. Relationships help reduce data duplication and maintain data integrity.

Exam Definition

A relationship is an association between two or more tables using a Primary Key and a Foreign Key.

Relationship Used in This Project

This shopping store database uses:

1. **One-to-Many (1:M)**
2. **Many-to-Many (M:N)**

Database Tables

The database contains **4 tables**:

1. Customers
2. Orders
3. Products
4. Order_Items

---

1. Customers Table

```
CREATE TABLE customers(
   cust_id SERIAL PRIMARY KEY,
   cust_name VARCHAR NOT NULL
);
```

Purpose

Stores customer information.

Primary Key

- cust_id

---

2. Orders Table

```
CREATE TABLE orders(
   ord_id SERIAL PRIMARY KEY,
   ord_date DATE NOT NULL,
   cust_id INT NOT NULL,
   FOREIGN KEY(cust_id)
   REFERENCES customers(cust_id)
);
```

Purpose

Stores order details.

Foreign Key

- cust_id references customers(cust_id).

---

3. Products Table

```
CREATE TABLE products(
   p_id SERIAL PRIMARY KEY,
   p_name VARCHAR(50) NOT NULL,
   price NUMERIC NOT NULL
);
```

Purpose

Stores product details.

Columns

- Product ID
- Product Name
- Product Price

---

4. Order_Items Table

```
CREATE TABLE ord_items(
   items_id SERIAL PRIMARY KEY,
   ord_id INT NOT NULL,
   p_id INT NOT NULL,
   quantity INT NOT NULL,
   FOREIGN KEY(ord_id)
      REFERENCES orders(ord_id),
   FOREIGN KEY(p_id)
      REFERENCES products(p_id)
);
```

Purpose

Stores the products included in each order.

It acts as a **junction (bridge) table** between **Orders** and **Products**.

One-to-Many Relationship

Customers → Orders

One customer can place **many orders**, but each order belongs to **only one customer**.

Customers

|

| 1

|

|------< Many

Orders

Example

Customer Orders

Sonu Order 1

Customer Orders

Sunil Order 2

Sunil Order 4

Customer **Sunil** has **multiple orders**.

Many-to-Many Relationship

Orders ↔ Products

One order can contain **many products**, and one product can appear in **many different orders**.

This relationship is implemented using the **Order_Items** table.

Orders

|

| One

|

Order_Items

|

| Many

|

Products

Example

Order 1

- Laptop
- Cable

Order 3

- Mouse
- Cable

Here, **Cable** appears in more than one order.

Why Use Order_Items?

Without the **Order_Items** table:

- One order could store only one product.
- One product could not belong to multiple orders.

The junction table solves this problem.

Entity Relationship Diagram (ERD)

Customers

---

cust_id (PK)

cust_name

|

| 1

|

|<--------------------*

Orders

---

ord_id (PK)

ord_date

cust_id (FK)

|

| 1

|

|<--------------------*

Order_Items

---

items_id (PK)

ord_id (FK)

p_id (FK)

quantity

|

| *--------------------1

|

Products

----

p_id (PK)

p_name

price

Inserting Data

Data is inserted into:

- Customers
- Orders
- Products
- Order_Items

using the INSERT INTO statement.

Example

```
INSERT INTO customers(cust_name)
VALUES
('Sonu'),
('Sunil'),
('Antara'),
('Alex');
```

Displaying Customer Orders with Product Details

```
SELECT
o.ord_id,
c.cust_name,
o.ord_date,
p.p_name,
oi.quantity,
p.price,
oi.quantity * p.price AS total_price

FROM ord_items oi

JOIN products p
ON oi.p_id=p.p_id

JOIN orders o
ON oi.ord_id=o.ord_id

JOIN customers c
ON c.cust_id=o.cust_id;
```

Output

Order ID	Customer	Order Date	Product	Quantity	Price	Total Price
1	Sonu	2024-01-01	Laptop	1	55000	55000
1	Sonu	2024-01-01	Cable	2	250	500
2	Sunil	2024-02-02	Laptop	1	55000	55000

Order ID	Customer	Order Date	Product	Quantity	Price	Total Price
3	Antara	2024-03-03	Mouse	1	400	400
3	Antara	2024-03-03	Cable	5	250	1250
4	Sunil	2024-04-01	Keyboard	2	700	1400

JOINs Used

The query uses **INNER JOIN** to combine data from four tables.

- Customers → Orders
- Orders → Order_Items
- Order_Items → Products

This displays complete order information.

Relationship Summary

Relationship	Tables
One-to-Many	Customers → Orders
One-to-Many	Orders → Order_Items
One-to-Many	Products → Order_Items
Many-to-Many	Orders ↔ Products (through Order_Items)

Advantages

- Eliminates duplicate data.
- Maintains data consistency.
- Supports multiple products in one order.
- Allows one product to appear in multiple orders.
- Makes querying and reporting easier.
- Follows database normalization principles.

Summary

- **Customers** store customer details.
- **Orders** store customer orders.
- **Products** store product information, including price.
- **Order_Items** connects orders and products using a many-to-many relationship.
- The relationship between **Customers** and **Orders** is **One-to-Many**.
- The relationship between **Orders** and **Products** is **Many-to-Many**, implemented through the **Order_Items** junction table.
- Using JOIN queries, we can display each customer, their orders, the products in each order, the quantity ordered, the product price, and the total price.

Interview / Exam Definition

In a shopping store database, a One-to-Many relationship exists between Customers and Orders, while a Many-to-Many relationship exists between Orders and Products. The Many-to-Many relationship is implemented using a junction table (Order_Items) that stores product quantities for each order.

PostgreSQL Views – Short Notes

What is a View?

A **View** is a **virtual table** created from one or more existing tables using a **SELECT** query. It does not store data itself; it displays data from the underlying tables.

Exam Definition

A View is a virtual table created using a SQL query that displays data from one or more tables without storing the data physically.

Why Use Views?

- Simplifies complex queries.
- Improves data security by showing only required columns.
- Reuses frequently used queries.
- Makes reports easier to generate.

- Hides the complexity of joins.

Syntax

```
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name
WHERE condition;
```

Example 1 – Billing Information View

```
CREATE VIEW billing_info AS
SELECT
   o.ord_id,
   c.cust_name,
   o.ord_date,
   p.p_name,
   oi.quantity,
   p.price,
   oi.quantity * p.price AS total_price
FROM ord_items oi
JOIN products p ON oi.p_id = p.p_id
JOIN orders o ON oi.ord_id = o.ord_id
JOIN customers c ON c.cust_id = o.cust_id;
```

Purpose

Displays complete billing information by combining data from:

- Customers
- Orders
- Products

- Order_Items

Display the View

SELECT * FROM billing_info;

Example 2 – Customer Purchase Summary

CREATE VIEW customer_buy AS

SELECT

  c.cust_name,

  SUM(oi.quantity * p.price) AS total_buy

FROM ord_items oi

JOIN products p ON oi.p_id = p.p_id

JOIN orders o ON oi.ord_id = o.ord_id

JOIN customers c ON c.cust_id = o.cust_id

GROUP BY c.cust_name;

Purpose

Displays the **total amount spent by each customer**.

Display the View

SELECT * FROM customer_buy;

Drop a View

DROP VIEW customer_buy;

Deletes the view only. The original tables and data remain unchanged.

List All Views

SELECT viewname

FROM pg_views;

Displays all views in the current database.

Advantages of Views

- Simplifies complex SQL queries.
- Improves security.
- Hides unnecessary columns.
- Reusable for reports.
- No duplicate data storage.

Limitations of Views

- A view does not store data.
- Data depends on the original tables.
- Deleting a view does not delete table data.

Summary

- A **View** is a **virtual table** created using a SELECT query.
- It can combine data from one or more tables.
- CREATE VIEW creates a view.
- SELECT * FROM view_name; displays the view.
- DROP VIEW removes the view.
- pg_views lists all available views.

Interview Definition

A View is a virtual table based on the result of a SELECT query. It stores only the query, not the actual data, and is used to simplify queries, improve security, and present data in a customized format.

HAVING Clause – Short Notes

What is HAVING?

The **HAVING** clause is used to **filter grouped data** after the GROUP BY clause. It is mainly used with **aggregate functions** such as SUM(), COUNT(), AVG(), MAX(), and MIN().

Exam Definition

The HAVING clause filters groups created by the GROUP BY clause based on aggregate function conditions.

Syntax

```
SELECT column_name,
aggregate_function(column_name)
FROM table_name
GROUP BY column_name
HAVING condition;
```

Example

```
SELECT p_name,
SUM(total_price)
FROM billing_info
GROUP BY p_name
HAVING SUM(total_price) > 1500;
```

Explanation

- Groups records by **product name**.
- Calculates the **total sales** of each product.
- Displays only products whose total sales are **greater than 1500**.

Difference Between WHERE and HAVING

WHERE	HAVING
Filters rows before grouping	Filters groups after grouping
Cannot use aggregate functions directly	Used with aggregate functions

WHERE	HAVING
Used before GROUP BY	Used after GROUP BY

GROUP BY ROLLUP – Short Notes

What is ROLLUP?

ROLLUP is an extension of the GROUP BY clause that generates **subtotals and a grand total** automatically.

Exam Definition

ROLLUP is used with GROUP BY to generate subtotal rows and a final grand total.

Syntax

```
SELECT column_name,
SUM(column_name)
FROM table_name
GROUP BY ROLLUP(column_name);
```

Example

```
SELECT
COALESCE(p_name,'Total'),
SUM(total_price) AS amount
FROM billing_info
GROUP BY ROLLUP(p_name)
ORDER BY amount;
```

Explanation

- Groups records by **product name**.
- Calculates the **total sales** of each product.
- ROLLUP adds an extra row showing the **grand total**.
- COALESCE() replaces the NULL value in the total row with **'Total'**.

Example Output

Product	Amount
Mouse	400
Cable	1750
Keyboard	1400
Laptop	110000
Total	**113550**

Advantages of ROLLUP

- Generates subtotals automatically.
- Generates a grand total.
- Useful for reports and sales summaries.
- Reduces the need for multiple SQL queries.

Summary

- **HAVING** filters grouped data based on aggregate conditions.
- **ROLLUP** generates subtotals and a grand total.
- **COALESCE()** is often used with ROLLUP to replace NULL with a meaningful label such as **'Total'**.

Interview Definitions

- **HAVING:** Filters grouped records after GROUP BY using aggregate function conditions.
- **ROLLUP:** An extension of GROUP BY that automatically generates subtotal and grand total rows.

Stored Routines in PostgreSQL – Complete Notes

What is a Stored Routine?

A **Stored Routine** is a collection of one or more SQL statements stored in the database server. It can be executed (called) multiple times whenever required.

Exam Definition

A Stored Routine is a set of SQL statements stored in the database that can be executed repeatedly to perform specific tasks.

Types of Stored Routines

There are two types of stored routines:

1. **Stored Procedure**
2. **User-Defined Function (UDF)**

---

1. Stored Procedure

What is a Stored Procedure?

A **Stored Procedure** is a collection of SQL statements and procedural logic that performs operations such as **INSERT, UPDATE, DELETE, or other database tasks**.

Unlike functions, procedures **do not have to return a value**.

Exam Definition

A Stored Procedure is a reusable set of SQL statements stored in the database that performs operations such as inserting, updating, deleting, or querying data.

Advantages of Stored Procedures

- Reusable code.
- Reduces code duplication.
- Faster execution.
- Improves security.
- Easy to maintain.

Syntax

```
CREATE OR REPLACE PROCEDURE procedure_name(
   parameters
```

```
)
LANGUAGE plpgsql
AS $$
BEGIN
   SQL statements;
END;
$$;
```

Example 1 – Update Employee Salary

Procedure

```
CREATE OR REPLACE PROCEDURE update_emp_salary(
   p_employee_id INT,
   p_new_salary NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
   UPDATE emp_info
   SET salary=p_new_salary
   WHERE emp_id=p_employee_id;
END;
$$;
```

Call Procedure

```
CALL update_emp_salary(3,90000);
```

Verify

```
SELECT * FROM emp_info;
```

Output

Before

EMP_ID FNAME SALARY

3 Arjun 55000

After

EMP_ID FNAME SALARY

3 Arjun 90000

Explanation

Employee ID **3** has its salary updated from **55000** to **90000**.

Example 2 – Insert New Employee

Procedure

```
CREATE OR REPLACE PROCEDURE add_employee(
   p_fname VARCHAR,
   p_lname VARCHAR,
   p_email VARCHAR,
   p_dept VARCHAR,
   p_salary NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
   INSERT INTO emp_info(
      fname,
      lname,
      email,
      dept,
      salary
```

```
    )
    VALUES(
        p_fname,
        p_lname,
        p_email,
        p_dept,
        p_salary
    );
END;
$$;
```

Call Procedure

```
CALL add_employee(
'Sunil',
'Behera',
'sunil.behera@gmail.com',
'IT',
30000
);
```

Verify

```
SELECT * FROM emp_info;
```

Output

EMP_ID	FNAME	LNAME	DEPT	SALARY
12	Sunil	Behera	IT	30000

A new employee is successfully inserted.

User-Defined Function (UDF)

What is a User-Defined Function?

A **User-Defined Function (UDF)** is a function created by the user to perform a specific task and **return a value or a table**.

Exam Definition

A User-Defined Function is a custom function created by the user to perform specific operations and return a value or a table.

Advantages of Functions

- Reusable logic.
- Returns values.
- Simplifies complex queries.
- Improves readability.
- Can be used inside SQL statements.

Syntax

```
CREATE OR REPLACE FUNCTION function_name(
   parameters
)
RETURNS datatype
AS $$
BEGIN
   SQL statements;
END;
$$
LANGUAGE plpgsql;
```

Example – Highest Salary Employee by Department

Function

```
CREATE OR REPLACE FUNCTION dept_max_sal_emp(
```

```
    dept_name VARCHAR
)
RETURNS TABLE(
    emp_id INT,
    fname VARCHAR,
    salary NUMERIC
)
AS $$
BEGIN
    RETURN QUERY
    SELECT
        e.emp_id,
        e.fname,
        e.salary
    FROM emp_info e
    WHERE e.dept=dept_name
    AND e.salary=(
        SELECT MAX(emp.salary)
        FROM emp_info emp
        WHERE emp.dept=dept_name
    );
END;
$$
LANGUAGE plpgsql;
```

Call Function

```
SELECT *
```

FROM dept_max_sal_emp('IT');

Output

EMP_ID	FNAME	SALARY
3	Arjun	90000

Returns the employee with the **highest salary** in the **IT** department.

RETURN QUERY

RETURN QUERY returns the result of a SQL query from a function.

Example

RETURN QUERY

SELECT *

FROM emp_info;

Procedure vs Function

Stored Procedure	User-Defined Function
Performs database operations	Returns a value or table
Called using CALL	Called using SELECT
Return value is optional	Must return a value or table
Used for INSERT, UPDATE, DELETE	Used for calculations and queries

---

Commands Used

Create Procedure

CREATE OR REPLACE PROCEDURE

Creates or replaces a stored procedure.

Call Procedure

CALL procedure_name();

Executes the procedure.

Create Function

CREATE OR REPLACE FUNCTION

Creates or replaces a function.

Execute Function

SELECT * FROM function_name();

Executes the function and returns the result.

Summary

- A **Stored Routine** is a reusable set of SQL statements stored in the database.
- There are **two types** of stored routines:
    - Stored Procedure
    - User-Defined Function
- A **Stored Procedure** performs operations such as **INSERT**, **UPDATE**, and **DELETE** and is executed using the CALL statement.
- A **User-Defined Function** performs a specific task and returns a value or table. It is executed using the SELECT statement.
- Stored routines improve performance, reduce code duplication, and simplify database programming.

Interview Definitions

Stored Routine

A stored routine is a collection of SQL statements stored in the database server that can be executed repeatedly.

Stored Procedure

A stored procedure is a reusable set of SQL statements that performs database operations such as inserting, updating, deleting, or querying data.

User-Defined Function

A user-defined function is a custom function created by the user to perform a specific task and return a value or a table.

Difference

- **Procedure:** Executes operations, called with CALL.
- **Function:** Returns data, called with SELECT.

Window Functions in PostgreSQL – Complete Notes

What are Window Functions?

Window Functions, also called **Analytic Functions**, perform calculations across a set of rows related to the current row **without grouping the rows**.

Unlike aggregate functions (SUM(), COUNT(), etc.), window functions **do not reduce the number of rows**. Every row remains visible in the output.

Exam Definition

Window Functions are SQL functions that perform calculations across a set of related rows while keeping each row separate. They are defined using the OVER() clause.

OVER() Clause

Every window function uses the **OVER()** clause.

Syntax

```
function_name() OVER(
   [PARTITION BY column_name]
   [ORDER BY column_name]
)
```

Explanation

- **OVER()** → Defines the window.
- **PARTITION BY** → Divides rows into groups.

- **ORDER BY** → Orders rows within each group.

Benefits of Window Functions

- Perform advanced analysis.
- Calculate running totals.
- Assign row numbers.
- Rank records.
- Compare current row with previous or next row.
- Preserve all rows in the result.

Sample EMP_INFO Table

Emp_ID	Name	Dept	Salary
1	Raj	IT	50000
2	Priya	HR	45000
3	Suman	Finance	60000
4	Kavita	HR	47000
5	Amit	Marketing	52000
6	Neha	IT	48000
7	Rahul	IT	53000
8	Anjali	Finance	61000
9	Vijay	Marketing	50000
10	Sonu	IT	300000
11	Arjun	IT	90000
12	Sunil	IT	30000

---

1. SUM() OVER()

Calculates a **running total**.

Syntax

```
SELECT column_name,
SUM(column_name)
OVER(ORDER BY column_name)
FROM table_name;
```

Example

```
SELECT
fname,
salary,
SUM(salary)
OVER(ORDER BY salary)
FROM emp_info;
```

Output (Example)

Name	Salary	Running Total
Sunil	30000	30000
Priya	45000	75000
Kavita	47000	122000
Raj	50000	172000

Explanation

Each row displays the cumulative salary up to that row.

---

2. ROW_NUMBER()

Assigns a unique row number to each row.

Syntax

```
SELECT
ROW_NUMBER()
```

OVER(ORDER BY column_name)

FROM table_name;

Example

SELECT

ROW_NUMBER()

OVER(ORDER BY fname),

fname,

salary

FROM emp_info;

Output

Row No	Name	Salary
1	Amit	52000
2	Anjali	61000
3	Arjun	90000

Explanation

Rows are numbered according to **alphabetical order of names**.

---

3. ROW_NUMBER() with PARTITION BY

Generates row numbers **within each department**.

Syntax

SELECT

ROW_NUMBER()

OVER(PARTITION BY column_name)

FROM table_name;

Example

SELECT

```
ROW_NUMBER()
OVER(PARTITION BY dept),
fname,
dept,
salary
FROM emp_info;
```

Output

Row	Department	Name
1	Finance	Anjali
2	Finance	Suman
1	HR	Kavita
2	HR	Priya
1	IT	Raj
2	IT	Rahul

Explanation

Row numbering starts again for every department.

---

4. RANK()

Assigns a rank based on ordering.

Duplicate values receive the **same rank**.

The next rank is skipped.

Syntax

```
SELECT
RANK()
OVER(ORDER BY salary DESC)
```

FROM emp_info;

Example

SELECT

fname,

dept,

salary,

RANK()

OVER(ORDER BY salary DESC)

FROM emp_info;

Example Output

Name	Salary	Rank
Sonu	300000	1
Arjun	90000	2
Anjali	61000	3
Suman	60000	4
Rahul	53000	5
Amit	52000	6
Raj	50000	7
Vijay	50000	7
Neha	48000	9

Explanation

Since Raj and Vijay have the same salary, both receive **Rank 7**, and **Rank 8 is skipped**.

---

5. DENSE_RANK()

Works like RANK() but **does not skip numbers**.

Syntax

```
SELECT
DENSE_RANK()
OVER(ORDER BY salary DESC)
FROM emp_info;
```

Example

```
SELECT
fname,
dept,
salary,
DENSE_RANK()
OVER(ORDER BY salary DESC)
FROM emp_info;
```

Output

Name	Salary	Dense Rank
Sonu	300000	1
Arjun	90000	2
Anjali	61000	3
Suman	60000	4
Rahul	53000	5
Amit	52000	6
Raj	50000	7
Vijay	50000	7
Neha	48000	8

Explanation

No rank is skipped.

Difference Between RANK() and DENSE_RANK()

RANK()	DENSE_RANK()
Skips rank after duplicates	Does not skip rank
Example: 1,2,2,4	Example: 1,2,2,3

---

6. LAG()

Returns the **previous row's value**.

Syntax

```
SELECT
LAG(column_name)
OVER(ORDER BY column_name)
FROM table_name;
```

Example

```
SELECT
fname,
dept,
salary,
LAG(salary)
OVER()
FROM emp_info;
```

Output

Name	Salary	Previous Salary
Raj	50000	NULL
Priya	45000	50000
Suman	60000	45000

Explanation

Displays the salary from the previous row.

---

7. LEAD()

Returns the **next row's value**.

Syntax

SELECT

LEAD(column_name)

OVER(ORDER BY column_name)

FROM table_name;

Example

SELECT

fname,

dept,

salary,

LEAD(salary)

OVER(ORDER BY salary)

FROM emp_info;

Output

Name Salary Next Salary

Sunil 30000 45000

Priya 45000 47000

Kavita 47000 48000

Explanation

Displays the salary of the next employee.

Example – Salary Difference

SELECT

```
fname,
dept,
salary,
(salary-
LEAD(salary)
OVER(ORDER BY salary DESC))
AS diff
FROM emp_info;
```

Explanation

Calculates the difference between the current employee's salary and the next highest salary.

Summary Table

Function	Purpose
SUM() OVER()	Running total
ROW_NUMBER()	Unique row numbering
RANK()	Ranking with skipped ranks
DENSE_RANK()	Ranking without skipped ranks
LAG()	Previous row value
LEAD()	Next row value

Advantages of Window Functions

- Perform running totals.
- Rank employees or products.
- Compare current, previous, and next rows.
- Preserve every row in the output.
- Useful for reports and analytics.

Interview Definitions

Window Function

A Window Function performs calculations across a set of related rows while keeping every row in the result set. It is defined using the **OVER()** clause.

OVER()

The OVER() clause defines the window (set of rows) on which a window function operates.

ROW_NUMBER()

Assigns a unique sequential number to each row.

RANK()

Assigns ranks to rows. Equal values receive the same rank, and the next rank is skipped.

DENSE_RANK()

Assigns ranks to rows. Equal values receive the same rank, but no ranks are skipped.

LAG()

Returns the value from the previous row.

LEAD()

Returns the value from the next row.

Common Table Expression (CTE) – Complete Notes

What is a Common Table Expression (CTE)?

A **Common Table Expression (CTE)** is a temporary result set created using the **WITH** clause. It exists only during the execution of a single SQL statement and helps make complex queries easier to read and manage.

Exam Definition

A Common Table Expression (CTE) is a temporary named result set created using the WITH clause that can be referenced within a SELECT, INSERT, UPDATE, or DELETE statement.

Why Use CTE?

- Simplifies complex SQL queries.

- Improves code readability.
- Avoids repeating the same subquery.
- Makes SQL easier to maintain.
- Can be used for recursive queries.

Syntax

```
WITH cte_name AS
(
   SELECT column_name
   FROM table_name
   WHERE condition
)

SELECT *
FROM cte_name;
```

Working Table

The following examples use the **employee** table.

emp_id	fname	dept	salary
101	Raju	Loan	37000
102	Sham	Cash	32000
103	Baburao	Loan	25000
104	Paul	Account	45000
105	Alex	Deposit	35000
106	Rick	Account	65000
107	Leena	Cash	25000

emp_id	fname	dept	salary
108	John	IT	75000
109	Alex	Loan	40000

Example 1 – Average Salary of Each Department

Problem

We want to calculate the **average salary** of each department and then display employees whose salary is **greater than their department's average salary**.

CTE Query

```
WITH avg_sal AS
(
   SELECT
      dept,
      ROUND(AVG(salary)) AS avg_salary
   FROM employee
   GROUP BY dept
)

SELECT
   e.emp_id,
   e.fname,
   e.salary,
   a.avg_salary
FROM employee e
JOIN avg_sal a
ON e.dept = a.dept
WHERE e.salary > a.avg_salary;
```

Explanation

Step 1

The CTE avg_sal calculates the average salary for each department.

Example

Department	Average Salary
Loan	34000
Cash	28500
Account	55000
Deposit	35000
IT	75000

Step 2

The main query joins the employee table with the CTE.

Step 3

The WHERE clause displays only employees whose salary is greater than their department's average.

Output

Emp_ID	Employee	Salary	Avg Salary
101	Raju	37000	34000
102	Sham	32000	28500
106	Rick	65000	55000
109	Alex	40000	34000

Example 2 – Highest Paid Employee in Each Department

Problem

We want to find the employee who has the **highest salary in each department**.

CTE Query

```
WITH high_salary AS
```

```
(
   SELECT
      dept,
      MAX(salary) AS max_salary
   FROM employee
   GROUP BY dept
)

SELECT
   e.emp_id,
   e.fname,
   e.lname,
   e.desig,
   e.dept,
   e.salary
FROM employee e
JOIN high_salary h
ON e.dept = h.dept
AND e.salary = h.max_salary;
```

Explanation

Step 1

The CTE high_salary calculates the highest salary in every department.

Example

Department	Maximum Salary
Loan	40000
Cash	32000

Department	Maximum Salary
Account	65000
Deposit	35000
IT	75000

Step 2

The main query joins the employee table with the CTE.

Step 3

The condition

e.salary = h.max_salary

returns only employees whose salary matches the highest salary in their department.

Output

Emp_ID	Employee	Designation	Department	Salary
109	Alex	Probation	Loan	40000
102	Sham	Cashier	Cash	32000
106	Rick	Manager	Account	65000
105	Alex	Associate	Deposit	35000
108	John	Manager	IT	75000

Advantages of CTE

- Improves readability.
- Simplifies complex SQL statements.
- Eliminates repeated subqueries.
- Makes queries easier to debug.
- Supports recursive queries.
- Enhances code maintenance.

CTE vs Subquery

CTE	Subquery
Created using WITH	Written inside the main query
Easy to read	Can become difficult for large queries
Can be reused within the same query	Must be repeated if needed multiple times
Better for complex queries	Better for simple queries

Summary

- A **Common Table Expression (CTE)** is a temporary named result set created using the **WITH** clause.
- It exists only during the execution of a single SQL statement.
- CTEs simplify complex queries and improve readability.
- In the first example, the CTE calculates the **average salary** for each department and displays employees earning above the department average.
- In the second example, the CTE calculates the **maximum salary** in each department and returns the highest-paid employee from every department.

Interview Definitions

Common Table Expression (CTE)

A **Common Table Expression (CTE)** is a temporary named result set created using the WITH clause that simplifies complex SQL queries and can be referenced within a single SQL statement.

WITH Clause

The **WITH clause** is used to define a Common Table Expression before executing the main SQL query.

Key Points

- Created using the WITH keyword.
- Exists only for the duration of one SQL statement.
- Improves readability and maintainability.
- Commonly used with SELECT, INSERT, UPDATE, and DELETE.
- Can be recursive using WITH RECURSIVE.

- **Once CTE has been created it can only be used once. It will not be persisted.**

Triggers in PostgreSQL – Complete Notes

What is a Trigger?

A **Trigger** is a special database object that automatically executes a function when a specific event (such as **INSERT, UPDATE, DELETE, or TRUNCATE**) occurs on a table or view.

Exam Definition

A Trigger is a special procedure in a database that automatically executes predefined actions in response to certain events on a specified table or view.

Why Use Triggers?

- Automatically validate data.
- Maintain data integrity.
- Keep audit logs.
- Prevent invalid data.
- Perform automatic calculations.
- Execute actions without user intervention.

Trigger Events

A trigger can be executed when the following events occur:

- **INSERT**
- **UPDATE**
- **DELETE**
- **TRUNCATE**

Trigger Timing

Trigger	Description
BEFORE	Executes before the event occurs.
AFTER	Executes after the event occurs.
INSTEAD OF	Used with views instead of the original operation.

General Syntax of Trigger

```
CREATE TRIGGER trigger_nam
{ BEFORE | AFTER | INSTEAD OF }
{ INSERT | UPDATE | DELETE | TRUNCATE }
ON table_name
FOR EACH { ROW | STATEMENT }
EXECUTE FUNCTION trigger_function_name();
```

Trigger Function Syntax

A trigger always executes a **Trigger Function**.

```
CREATE OR REPLACE FUNCTION trigger_function_name()
RETURNS TRIGGER AS $$
BEGIN
   -- Trigger Logic
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Explanation

- **RETURNS TRIGGER** → Indicates it is a trigger function.
- **BEGIN ... END** → Contains the trigger logic.
- **RETURN NEW** → Returns the modified/new row.

Example – Prevent Negative Salary

Problem

If a user inserts or updates a **negative salary**, the trigger should automatically change it to **0**.

Step 1 – Create Trigger Function

```
CREATE OR REPLACE FUNCTION check_salary()
RETURNS TRIGGER AS $$
BEGIN
   IF NEW.salary < 0 THEN
      NEW.salary := 0;
   END IF;
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Explanation

- Checks the new salary.
- If salary is negative, it changes it to **0**.
- Returns the updated row.

Step 2 – Create Trigger

```
CREATE TRIGGER before_insert_update_salary
BEFORE INSERT OR UPDATE
ON employee
FOR EACH ROW
EXECUTE FUNCTION check_salary();
```

Example

Insert

INSERT INTO employee

(emp_id,fname,lname,desig,dept,salary)

VALUES

(110,'Rahul','Patel','Manager','IT',-5000);

Output

emp_id	fname	salary
110	Rahul	0

The trigger automatically converts **-5000** into **0**.

Update Example

UPDATE employee

SET salary=-10000

WHERE emp_id=101;

Result

Salary becomes

0

instead of

-10000

Types of Triggers

Type	Description
BEFORE Trigger	Executes before an operation.
AFTER Trigger	Executes after an operation.

Type	Description
INSTEAD OF Trigger	Used on views instead of the actual operation.

Advantages of Triggers

- Automatic execution.
- Maintains data integrity.
- Prevents invalid data.
- Improves security.
- Useful for auditing.
- Reduces manual coding.

Summary

- A **Trigger** automatically executes when an event occurs on a table.
- A trigger always calls a **Trigger Function**.
- Common events are **INSERT**, **UPDATE**, **DELETE**, and **TRUNCATE**.
- Common timings are **BEFORE**, **AFTER**, and **INSTEAD OF**.
- Triggers are used for validation, auditing, and enforcing business rules.

CASCADE ON DELETE – Complete Notes

What is CASCADE ON DELETE?

ON DELETE CASCADE is a **Foreign Key** option. When a row in the **parent table** is deleted, PostgreSQL automatically deletes all related rows from the **child table**.

Exam Definition

ON DELETE CASCADE is a foreign key action that automatically deletes matching records in the child table when the related record in the parent table is deleted.

Why Use ON DELETE CASCADE?

- Maintains referential integrity.

- Prevents orphan records.
- Automatically removes related data.
- Reduces manual deletion.

Example Tables

Customers (Parent Table)

cust_id	name
101	Raju
102	Sham
103	Baburao

Orders (Child Table)

order_id	amount	cust_id
1	200	101
2	500	102
3	1000	101

Customer **101** has two orders.

Syntax

```
CREATE TABLE orders(
   ord_id SERIAL PRIMARY KEY,
   ord_date DATE,
   amount DECIMAL(10,2),
   cust_id INT,
   FOREIGN KEY(cust_id)
   REFERENCES customers(cust_id)
```

ON DELETE CASCADE

);

Example

Parent Table

DELETE FROM customers

WHERE cust_id=101;

Before Delete

Customers

cust_id	Name
101	Raju
102	Sham
103	Baburao

Orders

order_id	cust_id
1	101
2	102
3	101

After Delete

Customers

cust_id	name
102	Sham
103	Baburao

Orders

order_id cust_id

2 102

The orders with **cust_id = 101** are automatically deleted.

Without CASCADE

If **ON DELETE CASCADE** is not used:

DELETE FROM customers

WHERE cust_id=101;

PostgreSQL returns an error because the customer is referenced in the **orders** table.

Advantages of ON DELETE CASCADE

- Maintains data consistency.
- Automatically deletes related records.
- Prevents orphan records.
- Reduces manual work.
- Improves database integrity.

Summary

- **Trigger** automatically executes SQL code when an event occurs on a table.
- **Trigger Function** contains the logic executed by the trigger.
- **ON DELETE CASCADE** automatically deletes child records when the related parent record is deleted.
- Triggers help automate tasks and enforce business rules, while **CASCADE ON DELETE** maintains relationships and referential integrity between parent and child tables.

Interview Definitions

Trigger

A Trigger is a database object that automatically executes a predefined function when an INSERT, UPDATE, DELETE, or TRUNCATE event occurs on a table or view.

Trigger Function

A Trigger Function is a PL/pgSQL function that contains the logic executed automatically by a trigger.

ON DELETE CASCADE

ON DELETE CASCADE is a foreign key option that automatically deletes related records in the child table whenever the corresponding record in the parent table is deleted.